EVERY TREE FOR HOPE

REFORESTATION & COMMUNITY IMPACT PLATFORM

TECHNICAL SPECIFICATION

Technology Stack & System Architecture

A comprehensive specification of the platforms, frameworks, libraries and infrastructure components comprising the Every Tree for Hope information system.

DOCUMENT TITLE	Technology Stack & System Architecture Specification
DOCUMENT REFERENCE	ETFH-TS-001
VERSION	1.0 — Released
ISSUE DATE	20 July 2026
SYSTEM	Every Tree for Hope — Web Platform & Administration Suite
ARCHITECTURE	Server-rendered modular monolith (PHP 8.2 / Laravel 12)
CLASSIFICATION	Internal — Technical Documentation
PREPARED BY	Engineering

This document reflects the state of the codebase as verified at the issue date. All version numbers are taken from the committed dependency manifests and resolved lock files.

FRONT MATTER

Document Control & Contents

Purpose and scope

This specification enumerates and describes every significant technology employed in the construction, operation and maintenance of the Every Tree for Hope platform. It is intended for technical stakeholders — engineers joining the project, technical reviewers, auditors, and partner organisations conducting due diligence.

It covers the application runtime, framework layer, data persistence, security model, presentation layer, internationalisation, third-party integrations, containerised infrastructure and quality-assurance tooling. Organisational process, budget and non-technical programme delivery are out of scope.

Revision history

Version	Date	Summary of change	Status
1.0	20 Jul 2026	Initial released specification. Full stack inventory verified against committed manifests and lock files.	Released

Contents

SECTION 1

Executive Summary

Every Tree for Hope is a modular monolithic web platform built on PHP 8.2 and Laravel 12, delivering a trilingual public-facing reforestation website alongside a comprehensive role-based administrative back office powered by Filament 5.

Architectural position

The system deliberately adopts a **server-rendered modular monolith** rather than a decoupled single-page application. All HTML is generated on the server via the Blade templating engine; the administrative interface uses Livewire to deliver reactive behaviour without a client-side framework. This choice yields a single deployment artefact, a single security boundary, first-class SEO for public content, and a materially lower operational burden — all appropriate characteristics for a non-governmental organisation platform where content discoverability and maintainability outweigh client-side interactivity.

Defining characteristics

Trilingual by design

Full English, Dari and Pashto support with right-to-left layout handling, covering both static interface strings and database-resident content.

Granular authorisation

A 125-permission role-based access control model across 21 resource modules, enforced by 22 dedicated policy classes.

Financial transparency

Expenditure tracking with bulk spreadsheet ingestion, surfaced publicly through an auditable reporting page.

Impact measurement

Longitudinal tracking of trees planted, lost and surviving, with maintenance-visit logs and computed survival rates per planting event.

Platform at a glance

175	~31.7k	36
PHP SOURCE FILES	LINES OF OWN CODE	DATABASE TABLES
25	3	153
ADMIN RESOURCES	SUPPORTED LOCALES	MANAGED PACKAGES

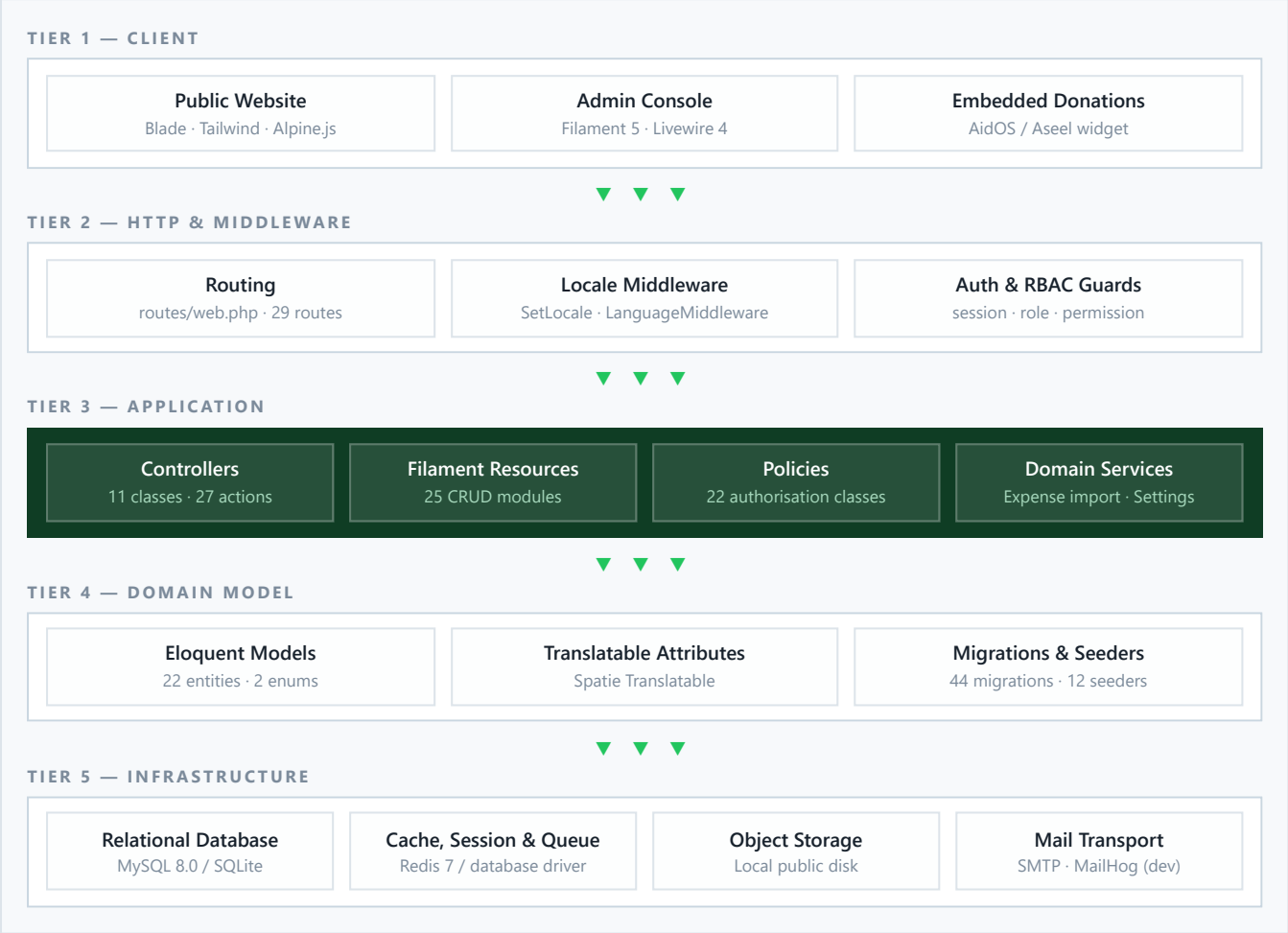
SUMMARY JUDGEMENT

The stack is modern, coherent and current — every primary framework runs on its latest major release. The architecture is well suited to the domain, and the authorisation subsystem is notably rigorous for a project of this scale. Section 16 records the outstanding remediation items required to bring the platform to full production readiness.

SECTION 2

System Architecture

The platform is organised into five logical tiers within a single deployable unit. Requests enter through a common HTTP kernel, are dispatched either to public Blade-rendered controllers or to the Livewire-driven administrative panel, and resolve against a shared Eloquent domain model over a single relational database.



Architectural principles observed

- **Single source of truth for permissions.** The entire permission catalogue is generated from one PHP class, eliminating drift between seeders, policies and the admin UI.
- **Convention-driven CRUD.** A shared `BasePolicy` supplies standard authorisation semantics; per-model policies remain thin subclasses.
- **Content as data.** Public pages read from database-backed, admin-editable entities rather than hard-coded markup, including a key/value site-settings store.
- **Outsourced payment surface.** No cardholder data traverses the application; donation capture is delegated entirely to a sandboxed third-party embed.

SECTION 3

Technology Stack Matrix

The following matrix consolidates every primary technology in the platform, organised by architectural concern. Version numbers denote the resolved release recorded in `composer.lock`, or the declared constraint where no lock file exists.

TABLE 3.1 — CONSOLIDATED TECHNOLOGY MATRIX

Layer	Technology	Version	Role in the system
Language	PHP	`8.2+`	Server-side runtime for all application logic
Framework	Laravel	`12.35.1`	HTTP kernel, routing, ORM, queues, validation, container
Admin panel	Filament	`5.2.0`	Complete administrative back office and CRUD engine
Reactivity	Livewire	`4.1.4`	Server-driven reactive components underpinning Filament
Templating	Blade	`Laravel 12`	Server-side rendering of all public pages
Authorisation	spatie/laravel-permission	`8.3.0`	Role and permission persistence and resolution
Translation	spatie/laravel-translatable	`6.11.4`	Per-locale JSON attributes on domain entities
Spreadsheets	PhpSpreadsheet	`5.5.0`	Bulk XLSX/CSV expense ingestion
Rich text	tiptap-php / CommonMark	`2.1.0 / 2.7.1`	Editor content serialisation and Markdown rendering
HTTP client	Guzzle	`7.10.0`	Outbound HTTP transport
Date handling	Carbon	`3.10.3`	Temporal arithmetic and localisation
Logging	Monolog	`3.9.0`	Structured application logging
Icons	blade-heroicons	`2.6.0`	SVG icon set for admin and views
Styling	Tailwind CSS	`4.x`	Utility-first design system
Client scripting	Alpine.js	`via Livewire`	Lightweight declarative DOM behaviour
Build tool	Vite	`7.x`	Asset bundling, hashing and HMR
Build bridge	laravel-vite-plugin	`2.x`	Manifest integration between Vite and Blade
Database	MySQL / SQLite	`8.0 / 3.x`	Primary relational persistence (per environment)
Cache & queue	Redis	`7-alpine`	Cache store, session store and queue backend
Containerisation	Docker & Compose	`3.8 schema`	Reproducible five-service development environment
Web server	Apache HTTP Server	`php:8.2-apache`	Request serving within the container image
Testing	PHPUnit	`11.5.42`	Unit and feature test execution
Mocking	Mockery	`1.6.12`	Test doubles and expectation mocking
Code style	Laravel Pint	`1.25.1`	Automated PHP formatting to PSR-12
Test data	FakerPHP	`1.24.1`	Seeded and factory-generated fixtures
Diagnostics	Pail / Tinker / Collision	`1.2.3 / 2.10.1 / 8.8.2`	Log tailing, REPL and error presentation

SECTION 4

Application Runtime & Backend Framework

4.1 PHP 8.2

The application targets PHP 8.2 or later, declared as `"php": "^8.2"` in the Composer manifest. The codebase makes use of modern language features including backed enumerations (`App\Enums\ExpenseType`, `App\Enums\PartnerType`), constructor property promotion, readonly semantics, named arguments and typed properties.

The container image installs the following PHP extensions: `pdo` `pdo_mysql` `mbstring` `exif` `pcntl` `bcmath` `gd` `zip` `intl`

4.2 Laravel 12

Laravel 12 (resolved `v12.35.1`) provides the application skeleton. The project adopts the **slim bootstrap** introduced in Laravel 11, configuring the entire HTTP kernel, middleware stack, route registration and exception handling from a single `bootstrap/app.php` file rather than dedicated kernel classes.

Framework subsystems in active use

Subsystem	Application within the platform
Routing	29 web route registrations; API route file intentionally absent
Eloquent ORM	22 models with accessors, casts, relationships and traits
Migrations	44 versioned schema migrations spanning Feb–Jul 2026
Validation	Controller-level request validation on all public form submissions
Authorisation gates	Policy registration plus a `Gate::before` super-administrator bypass
Middleware	Two global custom middleware plus three Spatie authorisation aliases
Filesystem	Public disk abstraction for CV, résumé, photo and media uploads
Queues	Database-backed in development, Redis-backed under Docker
Localisation	Locale resolution, translation catalogues and fallback chain
Service container	Provider-based binding of policies and panel configuration
Health endpoint	`/up` exposed for external liveness probing

4.3 HTTP interface design

The platform deliberately exposes **no REST or GraphQL API**. There is no `routes/api.php`, no token-issuing package (Sanctum or Passport), and no schema-definition layer. All interaction occurs through server-rendered HTML pages and standard form submissions.

The public surface comprises 20 `GET` endpoints, 7 `POST` endpoints and 2 permanent redirects, served by 11 controllers exposing 27 public actions. The administrative surface is generated automatically by Filament across 25 resources.

> **DESIGN RATIONALE**
> Omitting an API layer removes an entire class of authentication, rate-limiting and versioning concerns. Should a mobile client or partner integration be required in future, Laravel Sanctum can be introduced against the existing Eloquent models with no restructuring of the domain.

SECTION 5

Administrative Platform

5.1 Filament 5

The entire back office is built on **Filament 5.2.0**, a component-driven administration framework for Laravel. It is configured through `AdminPanelProvider`, which registers a panel identified as `admin` mounted at the `/admin` path, authenticated against the standard `web` guard and branded as *Every Tree for Hope Admin* with an amber primary palette.

Resources, pages and widgets are auto-discovered from their conventional namespaces, so new administrative modules require no manual registration.

Filament component packages in use

`filament/forms` `filament/tables` `filament/actions` `filament/infolists` `filament/notifications` `filament/schemas` `filament/support` `filament/widgets` — all at `v5.2.0`.

5.2 Navigation taxonomy

The 25 administrative resources are organised into six navigation groups, reflecting the operational divisions of the organisation:

Navigation group	Resources administered
Content Management	Events, Event Images, Upcoming Events, Media, Teams, FAQs, Partners
Careers	Companies, Job Categories, Job Postings, Job Applications
User Engagement	Voices, Voice Comments, Contact Messages, Tree Requests, Involvement Requests
Financial	Donators, Expenses, Sponsor Packages
System	Site Settings, application configuration
Access Control	Users, Roles, Permissions

5.3 Dashboard widgets

Five analytical widgets present operational state on the administrative landing page. Financial and activity widgets are individually gated behind dedicated permissions, so lower-privilege roles receive a reduced dashboard rather than an authorisation error.

Widget	Type	Purpose
OverviewStatsWidget	Stat cards	Headline platform counters
RecentDonationsChart	Chart.js chart	Donation volume over time
TreePlantingProgressWidget	Chart.js chart	Cumulative planting progress
RecentActivityWidget	Table	Latest submissions and changes
TopDonatorsWidget	Table	Highest-contributing supporters

5.4 Livewire 4

Filament is built upon **Livewire 4.1.4**, which supplies the reactive component model. Livewire maintains component state on the server and exchanges only DOM diffs with the browser, allowing rich interactive behaviour — live validation, dependent selects, modal actions, real-time table filtering — without a JavaScript application framework or a separate API contract.

SECTION 6

Presentation Layer

6.1 Blade templating

The public website comprises 44 Blade templates, of which 34 are project-authored. Templates are organised into layouts, partials, page views and components. Blade compiles views to plain PHP once and caches them, incurring no per-request parsing cost.

6.2 Tailwind CSS 4

Styling follows a **utility-first** methodology using Tailwind CSS 4, built through `@tailwindcss/vite` — the CSS-first plugin that removes the need for a JavaScript configuration file.

Brand design tokens

Token	Value	Application
primary	`#22c55e`	Principal brand green — actions and accents
deep-green	`custom`	Headers, footers and high-contrast surfaces
vibrant-lime	`custom`	Highlights and interactive states
sage-green	`custom`	Muted backgrounds and secondary surfaces
gold-accent	`custom`	Sponsorship and recognition emphasis
charcoal	`custom`	Body copy and neutral text

> **CONFIGURATION OBSERVATION**
> Two Tailwind delivery mechanisms are presently wired concurrently: the Vite-compiled pipeline, and the Tailwind Play CDN referenced from the primary layout. The Play CDN is not intended for production use; consolidation is recorded as item TD-01 in Section 16.

6.3 Typography

The type system pairs Latin faces with dedicated Arabic-script faces for Dari and Pashto.

Typeface	Script	Role
Manrope	Latin	Primary interface and body text
Playfair Display	Latin	Editorial display headings
Caveat	Latin	Handwritten accent for testimonial content
Vazirmatn	Arabic / Persian	Dari and Pashto interface text
Noto Naskh Arabic	Arabic	Extended Arabic-script coverage
Material Symbols Outlined	Iconographic	Public-site icon system

6.4 Client-side behaviour

Client-side JavaScript is intentionally minimal — the project's own script source totals fewer than ten lines. **Alpine.js**, delivered within the Livewire bundle, supplies declarative interactivity directly in markup through `x-data`, `x-show`, `x-ref` and `x-transition` directives, used for menus, disclosure panels and the donation embed. No single-page application framework is present: React, Vue, Svelte, Angular and Inertia are all deliberately absent, and no client-side state-management or routing library is required.

SECTION 7

Data Architecture & Persistence

7.1 Database engines

The platform is engine-agnostic by virtue of Laravel's database abstraction, and is configured differently per environment:

Environment	Engine	Configuration source
Local development	SQLite 3	`.env.example` sets `DB_CONNECTION=sqlite`; a database file is present
Containerised	MySQL 8.0	`docker-compose.yml` provisions the `mysql:8.0` image
Automated testing	SQLite in-memory	`phpunit.xml` forces `:memory:` for isolation and speed
Framework default	MySQL	`config/database.php` falls back to the MySQL driver

MySQL 8.0 is the intended production engine, consistent with the container image, which compiles only the `pdo_mysql` driver.

7.2 Object-relational mapping

Persistence is mediated entirely through **Eloquent**, Laravel's active-record ORM. The domain comprises 22 model classes plus a reusable `HasSponsorCode` trait that generates unique supporter reference codes. Models make extensive use of attribute casting, computed accessors, and both one-to-many and many-to-many relationships.

Notable computed attributes

Model	Accessor	Derivation
Event	`trees_alive`	Trees planted less trees lost
Event	`survival_rate`	Surviving trees as a proportion of trees planted
Media	`youtube_video_id`	Identifier parsed from a supplied YouTube URL
Media	`is_short`	Detection of short-form video format
Media	`thumbnail_url`	Derived preview image endpoint

7.3 Schema management

The schema is defined across **44 versioned migrations** dated between February and July 2026, producing **36 tables**. Reference and demonstration data are supplied by 12 seeder classes, including a dedicated `RolesAndPermissionsSeeder` that reconstructs the entire authorisation catalogue idempotently.

44	36	22
MIGRATIONS	TABLES	ELOQUENT MODELS

7.4 Domain entity inventory

The following table enumerates the persisted domain entities and their functional responsibility. Framework-internal tables (sessions, cache, jobs, failed jobs, batches, password reset tokens) are omitted.

TABLE 7.1 — DOMAIN ENTITIES

Entity	Domain	Responsibility
Event	Impact	Planting events with location, counts, survival and maintenance data
EventImage	Impact	Photographic gallery attached to a planting event
UpcomingEvent	Engagement	Scheduled future activities open to volunteer registration
Donator	Financial	Supporter records, contribution values and sponsor codes
SponsorPackage	Financial	Priced sponsorship tiers with fund-allocation breakdowns
Expense	Financial	Itemised expenditure underpinning public transparency reporting
Partner	Relationships	Institutional partners and advisors, typed by category
Team	Relationships	Staff and volunteer profiles
Company	Careers	Employing organisations for job listings
JobCategory	Careers	Translatable classification of vacancies
JobPosting	Careers	Translatable vacancy with employment type and experience level
JobApplication	Careers	Candidate submissions with résumé attachments
TreeRequest	Community	Community-originated planting requests with review workflow
InvolvementRequest	Community	Volunteer, sponsor and collaboration enquiries with CV upload
Voice	Community	Moderated community posts on the “Voices of Nature” wall
VoiceComment	Community	Threaded responses subject to moderation
VoiceLike	Community	Anonymous appreciation, deduplicated by fingerprint
ContactMessage	Communication	General enquiries received through the contact form
Faq	Content	Translatable frequently asked questions
Media	Content	Video story library backed by YouTube references
Setting	System	Key/value configuration store editable by administrators
User	Access control	Administrative accounts with assigned roles

Associative tables

`event_donator` attributes supporters to the specific plantings they funded, and `event_partner` records institutional involvement per event. Spatie contributes five further tables governing roles, permissions and their assignments.

7.5 Structured attributes

Several entities employ JSON-typed columns for naturally variable data: volunteer name lists and custom tree species on events, maintenance visit logs and photographs, sponsorship fund allocation breakdowns, and uploaded media paths on tree requests. Translatable attributes are likewise stored as per-locale JSON maps.

SECTION 8

Security & Access Control

The authorisation subsystem is the most rigorously engineered component of the platform, implementing a complete role-based access control model with a single authoritative permission catalogue and comprehensive policy coverage.

8.1 Authentication

Authentication uses Laravel's **session-based** mechanism with a single `web` guard backed by an Eloquent user provider. Credentials are hashed using bcrypt with a configurable work factor, applied automatically through the model's `hashed` attribute cast. The login interface is provided by Filament at `/admin/login`.

There is **no public user registration**. Accounts are created exclusively by administrators, which materially reduces the platform's authentication attack surface. Token, OAuth and JWT mechanisms are not present and are not required by the architecture.

8.2 Authorisation model

Authorisation is implemented on `spatie/laravel-permission 8.3.0`, extended by a project-specific architecture documented in `docs/RBAC.md`.

Permission catalogue

All permissions derive from a single class, `App\Support\PermissionCatalog`, which generates the complete set programmatically. This single-source-of-truth design guarantees that seeders, policies and the administrative interface can never diverge.

Permission family	Cardinality	Composition
Model abilities	21 modules × 6	`view_any`, `view`, `create`, `update`, `delete`, `delete_any`
Workflow abilities	2	`approve_voice`, `reject_voice`
Export abilities	2	`export_donator`, `export_expense`
Interface abilities	4	`view_dashboard`, `manage_site_settings`, `view_financial_widgets`, `view_activity_widgets`
Total	**Approximately 125 discrete permissions**	

Policy enforcement

Enforcement is delegated to **22 policy classes**. A single `BasePolicy` encodes the naming convention that maps an ability and model to its permission string; the 21 per-model policies are thin subclasses. Policies are registered centrally in `AuthServiceProvider`, which additionally installs a `Gate::before` callback granting the Super Administrator role unconditional access.

> **ARCHITECTURAL STRENGTH**
> Deriving 125 permissions and 22 policies from one convention means adding a new administered entity requires only a catalogue entry and a three-line policy subclass. The design scales without introducing authorisation gaps.

8.3 Role hierarchy

Five roles are provisioned by the `RolesAndPermissionsSeeder`, forming a graduated privilege ladder:

Role	Intended scope of authority
Super Admin	Unrestricted access via gate bypass; reserved for system custodians
Admin	Full operational administration across all modules
Manager	Content, engagement and financial management without access-control rights
Staff	Day-to-day content maintenance and submission processing
Viewer	Read-only visibility for auditors and observers

Panel entry is gated by `User::canAccessPanel()`, which admits only accounts holding at least one assigned role. A legacy boolean administrator column was removed by migration in July 2026 in favour of this role-driven check.

8.4 Framework-level protections

Control	Implementation
CSRF protection	Token verification on all state-changing form submissions
SQL injection	Parameter binding throughout the Eloquent query builder
Output escaping	Blade escapes interpolated output by default
Password storage	bcrypt hashing with configurable cost factor
Session security	Configurable encryption, lifetime, domain and path scoping
Mass assignment	Explicit fillable declarations on all models
Upload handling	Validated file uploads written to namespaced storage directories
Third-party embed	Donation provider rendered inside a sandboxed iframe
Abuse mitigation	Anonymous engagement deduplicated by 64-character fingerprint
Content moderation	Community submissions default to a pending state pending review

8.5 Payment data posture

The platform processes **no payment instruments whatsoever**. Donation capture is delegated entirely to the AidOS / Aseel embedded widget, which executes in an isolated browsing context. No cardholder data enters the application boundary, its database, or its logs. This materially reduces the compliance perimeter.

> **SECURITY REVIEW ITEMS**
> Two configuration matters warrant attention before production deployment: an environment file containing development credentials is committed to version control, and database passwords are declared inline in the Compose manifest. Both are development-scope values, and both are tracked as remediation items TD-02 and TD-03 in Section 16.

SECTION 9

Internationalisation & Localisation

Multilingual capability is a first-class architectural concern rather than a retrofitted feature. The platform serves three languages spanning two writing directions, covering both interface strings and administrator-authored content.

9.1 Supported locales

Code	Language	Direction	Script and typeface
`en`	**English**	Left to right	Latin — Manrope, Playfair Display
`fa`	**Dari (Afghan Persian)**	Right to left	Perso-Arabic — Vazirmatn
`ps`	**Pashto**	Right to left	Perso-Arabic — Vazirmatn, Noto Naskh Arabic

9.2 Two-tier translation strategy

The platform distinguishes between *static* interface text and *dynamic* administrator-authored content, applying a purpose-built mechanism to each.

Tier 1 — Interface strings

Laravel translation catalogues at `resources/lang/{en,fa,ps}/messages.php`, totalling 3,241 lines. Resolved at render time with a configured fallback chain.

Tier 2 — Database content

`spatie/laravel-translatable 6.11.4` stores per-locale values in JSON columns, allowing editors to maintain all three languages from a single admin form.

Translatable entities

Per-locale attributes are enabled on `upcoming_events`, `faqs`, `job_postings` and `job_categories`.

9.3 Locale resolution

Two global middleware components participate in every request. `SetLocale` establishes the active application locale, and `LanguageMiddleware` propagates presentation concerns derived from it. Selection persists across requests, and the resolved locale determines the document's `dir` attribute, the active typeface stack and the directional layout applied by Tailwind.

9.4 Right-to-left support

Right-to-left rendering is handled at the document level rather than through duplicated stylesheets. Setting the `dir` attribute on the root element allows Tailwind's logical properties to mirror margins, padding, alignment and flow automatically, while the typeface stack is exchanged for Arabic-script faces. Translator guidance is maintained in `TRANSLATION_GUIDE.md`.

> **OPERATIONAL SIGNIFICANCE**
>
> Serving Dari and Pashto natively, with correct right-to-left typography, is essential to the organisation's mission of engaging Afghan communities directly. The translation architecture permits new locales to be added without schema migration.

SECTION 10

Third-Party Integrations

10.1 Active integrations

TABLE 10.1 — INTEGRATIONS IN PRODUCTION USE

Integration	Provider / library	Function
Donation processing	AidOS / Aseel	Sandboxed campaign iframe plus hosted widget scripts; the destination URL is administrator-configurable through the settings store
Spreadsheet ingestion	PhpSpreadsheet 5.5.0	Bulk import of expenditure records from XLSX and CSV files, with date normalisation, numeric cleaning and expense-type inference
Video hosting	YouTube	Embedded video stories with identifier parsing, short-form detection and derived thumbnails
Web typography	Google Fonts	Delivery of the six-face multilingual type system
Data visualisation	Chart.js via Filament	Dashboard charts for donation volume and planting progress
Mapping	Embedded map markup	Per-event embed snippets stored as content; no SDK dependency
Social presence	Outbound links	Facebook, Instagram, LinkedIn, YouTube and WhatsApp referrals

10.2 Configured but not yet exercised

The following capabilities are configured within the framework and available for immediate use, but have no invoking application code at present:

- **Transactional email.** Mail transport is configured, with MailHog provisioned for local capture and driver scaffolding present for Postmark, Resend and Amazon SES. No mailable classes have yet been authored.
- **Amazon S3 storage.** Credentials and bucket variables are declared; the active filesystem disk remains local.
- **Slack notifications.** Service configuration is present without a consuming notification channel.

10.3 Deliberately excluded

The following categories of dependency are absent by design, and their exclusion should be read as an architectural decision rather than an omission:

Category	Rationale for exclusion
Native payment gateway	Payment handling is fully delegated, keeping the compliance perimeter minimal
Real-time / WebSockets	No feature requires server-pushed updates; broadcasting remains inert
Search engine	Dataset volume is well within the capability of indexed relational queries
SMS gateway	Engagement channels are web and email based
Server-side PDF generation	Published documents are prepared externally and served as static assets

> **RECOMMENDED ADDITION**
>
> Application performance monitoring and error aggregation are not currently instrumented. Introducing an exception-tracking service is recorded as remediation item TD-06.

SECTION 11

Infrastructure, Containerisation & Build Pipeline

11.1 Container image

The application image derives from `php:8.2-apache`. The build installs the required PHP extensions, copies Composer from its official image, and executes `composer install --no-dev --optimize-autoloader` to produce a production-optimised autoloader. The Apache document root is rewritten to the `public` directory so that only web-accessible assets are exposed.

11.2 Development environment topology

A five-service Docker Compose stack provides a reproducible development environment, isolated on a dedicated bridge network with named volumes for durable state.

TABLE 11.1 — COMPOSE SERVICES

Service	Image	Host port	Function
app	`local build`	`8000`	Apache and PHP application runtime
mysql	`mysql:8.0`	`33066`	Primary relational database
redis	`redis:7-alpine`	`63799`	Cache, session and queue backend
phpmyadmin	`phpmyadmin`	`8080`	Database administration interface
mailhog	`mailhog`	`1025 / 8025`	SMTP capture and inspection interface

Persistent volumes `mysql_data` and `redis_data` retain state across container recreation. Convenience wrappers `docker-commands.sh` and `docker-commands.bat` provide parity across POSIX and Windows workstations, with procedures documented in `DOCKER-README.md`.

11.3 Asset build pipeline

Front-end assets are compiled by **Vite 7** through `laravel-vite-plugin 2`, which writes a manifest consumed by Blade's `@vite` directive to resolve content-hashed filenames. Two entry points are declared: the application stylesheet and the application script. Hot module replacement with automatic Blade refresh is enabled during development.

11.4 Unified development workflow

A single Composer script starts the complete development environment concurrently, removing the need to coordinate multiple terminals:

Process	Responsibility
artisan serve	PHP development HTTP server
artisan queue:listen	Background job worker with automatic code reloading
artisan pail	Real-time formatted log streaming
npm run dev	Vite development server with hot module replacement

> **PIPELINE GAP**
> No continuous integration configuration is present in the repository. Establishing an automated pipeline that runs the test suite and code-style verification on every push is recorded as remediation item TD-04.

SECTION 12

Quality Assurance & Code Standards

12.1 Automated testing

The test suite is executed by **PHPUnit 11.5.42**, configured through `phpunit.xml` with two suites — `Unit` and `Feature`. The testing environment is fully isolated: an in-memory SQLite database, array-backed cache and session stores, a synchronous queue driver and a null mail transport ensure that no test can reach external state.

Test suite	Files	Coverage focus
Feature — Authorization	3	Panel access, role management and permission resolution
Framework stubs	2	Default scaffolding, retained but not extended
Base test case	1	Shared application bootstrapping

Test coverage is currently concentrated entirely on the access-control subsystem — appropriately, since that is the highest-risk area of the platform. Controllers, domain models, form validation and the expense import service are not yet covered. Expanding coverage is recorded as remediation item TD-05.

12.2 Code style

Laravel Pint 1.25.1 enforces PHP formatting. No custom rule set is defined, so the tool applies its default PSR-12-derived preset, giving a consistent and conventional style across the codebase with no configuration burden.

12.3 Developer tooling

Tool	Version	Purpose
Laravel Tinker	`2.10.1`	Interactive REPL against the live application container
Laravel Pail	`1.2.3`	Human-readable real-time log tailing
Nuno Collision	`8.8.2`	Structured console error reporting
Laravel Sail	`1.46.0`	Alternative Docker development harness
Mockery	`1.6.12`	Test doubles and behavioural expectations
FakerPHP	`1.24.1`	Realistic fixture generation for seeders and factories
Concurrently	`9.x`	Parallel orchestration of development processes

12.4 Project documentation

Document	Content
docs/RBAC.md	Access-control architecture, a ten-point security review and a deployment checklist
DOCKER-README.md	Container environment setup and operational procedures
TRANSLATION_GUIDE.md	Guidance for maintaining the three translation catalogues
README.md	Currently the unmodified framework template; superseding it is item TD-07

SECTION 13

Functional Module Inventory

Every Tree for Hope is a reforestation and community-impact platform serving Afghan communities. The system combines a public engagement website with a comprehensive operational back office across the following functional modules.

13.1 Impact & planting operations

The central module records planting events with title, description, location, province, embedded map reference, date, event type and video reference. Quantitative impact is captured as trees planted, trees lost and volunteers engaged, from which surviving-tree counts and survival rates are derived automatically.

The module extends beyond initial planting into **longitudinal stewardship**: each event maintains a last-maintenance timestamp, free-text maintenance notes, a structured log of maintenance visits and an associated photographic record. Volunteer names and tree species, including custom species entries, are stored as structured lists. Photographic galleries attach to each event through a dedicated image entity.

13.2 Financial transparency

Expenditure is recorded line by line with date, description, quantity, unit price, total cost, paying party, categorised expense type and supporting notes. Records may be entered individually or ingested in bulk from spreadsheets through a dedicated import service that normalises dates, cleans numeric formatting, extracts the paying party from column headers and infers expense categories. The resulting dataset drives a publicly accessible transparency report.

13.3 Supporters & sponsorship

Supporter records capture name, stated impact, location and location type, financial contribution, contact details, profile image, donation date and trees sponsored, together with a verification status and a featured flag for public recognition. Each supporter receives a unique sponsor code generated automatically. Supporters are attributed to the specific planting events their contributions funded.

Sponsorship tiers are modelled as configurable packages with title, price, currency (US dollar, euro or Afghani), tree count, descriptive copy, an optional badge label, and a structured allocation breakdown expressing how contributions are apportioned. These packages populate the public funding-model page.

13.4 Community request workflows

Two structured intake workflows accept community submissions, each governed by a five-state review lifecycle spanning pending, reviewing, approved, rejected and completed, with administrator notes and a review timestamp.

- **Tree requests** allow communities to petition for plantings, capturing location, requested tree count, available water source, responsible person, contact number and supporting photographs.
- **Involvement requests** handle volunteer, sponsor and collaboration enquiries, including curriculum vitae uploads and optional association with a scheduled upcoming event.

13.5 Voices of Nature

A moderated community wall invites the public to contribute ideas, findings, experiences and questions. Posts carry a slug for stable addressing, author name, email and country, an optional image, and denormalised counters for likes, comments and views. Every submission enters a moderation queue, governed by dedicated approve and reject permissions.

Engagement is deliberately frictionless: readers may express appreciation and comment **without registering an account**, with duplicates prevented by a 64-character fingerprint rather than by identity — preserving anonymity while limiting abuse.

13.6 Careers

A recruitment module spans four related entities. Employing organisations are recorded as companies; vacancies are classified by translatable categories and published as translatable postings carrying employment type, experience level, slug-based routing and an optional external application URL. Applications are captured with résumé attachments.

13.7 Engagement & content

Module	Capability
Upcoming Events	Translatable announcements of forthcoming activities, linked to volunteer registrations
Partners & Advisors	Typed institutional relationships with logo, description, sponsor code and per-event linkage; surfaced through separate public pages
Team	Staff and volunteer profiles with photograph, personal message and social link
Media library	Curated video stories and short-form content sourced from YouTube
Contact	Public enquiry form feeding an administrative inbox
FAQ	Translatable question-and-answer catalogue with public submission
Children's Awareness	Dedicated educational programme content for younger audiences
Editorial pages	About, Our Works, History, Report and Funding Model

13.8 Public route surface

TABLE 13.1 — REPRESENTATIVE PUBLIC ENDPOINTS

Method	Path	Function
GET	/	Landing page
GET	/event, /event/{event}	Planting event index and detail
GET	/report	Financial transparency report
GET	/funding-model	Sponsorship packages and allocation
GET	/donators, /partners, /advisors	Recognition listings
GET	/careers, /careers/{job}	Vacancy index and detail
GET	/voices, /voices/{voice}	Community wall index and detail
POST	/voices/{voice}/like, /comment	Anonymous engagement actions
POST	/tree-request	Community planting request intake
POST	/get-involved	Volunteer and sponsorship enquiry intake
POST	/contact	General enquiry submission

SECTION 14

Codebase Metrics

The following measurements characterise the scale and composition of the project's own source code; vendored dependencies are excluded throughout.

14.1 Source composition

TABLE 14.1 — LINES OF CODE BY AREA

Area	Files	Lines
Application code (app/)	175	10,839
Blade templates	44	12,409
Database layer	57	3,277
Translation catalogues	3	3,241
Configuration, routes and tests	—	1,881
JavaScript and CSS sources	3	16
Total project source	—	≈ 31,700

14.2 Structural inventory

TABLE 14.2 — COMPONENT COUNTS

Component	Count	Note
Eloquent models	22	Plus one shared trait
Database tables	36	Including framework and RBAC tables
Migrations	44	Feb–Jul 2026
Seeders / factories	12 / 1	Includes idempotent RBAC seeder
Filament resources	25	Across six navigation groups; plus 2 pages and 5 widgets
Authorisation policies	22	One base plus 21 subclasses
Discrete permissions	≈ 125	Generated from a single catalogue
Roles	5	Super Admin through Viewer
Controllers / actions	11 / 27	Public-facing website only
Web routes	29	Excluding generated admin routes
Backed enumerations	2	Expense type, partner type
Supported locales	3	One LTR, two RTL
Composer packages	118 / 35	Production / development
Static public assets	107	Images, documents and icons

14.3 Observations

The ratio of template to application code — roughly 12,400 lines of Blade against 10,800 of PHP — confirms the content-rich, server-rendered posture. The near-total absence of project-authored JavaScript shows how completely Livewire and Alpine displace conventional front-end tooling.

SECTION 15

Technology Selection Rationale

This section records the reasoning behind the principal technology decisions, together with the alternatives considered and the trade-offs accepted.

TABLE 15.1 — DECISION REGISTER

Decision	Alternative	Rationale and accepted trade-off
Modular monolith	Microservices	A single deployable artefact matches the organisation's operational capacity. Distributed architecture would impose coordination and observability costs disproportionate to the domain.
Server-rendered Blade	React or Vue SPA	Public content requires first-class indexability and fast initial paint. Server rendering eliminates hydration cost and an entire API contract. Rich client interactivity is forgone — not required here.
Livewire and Alpine	Client-side framework	Reactive administrative interfaces are achieved without a JavaScript build for behaviour, keeping business logic in one language and one codebase.
Filament admin	Bespoke admin	Twenty-five fully featured CRUD modules with search, filtering, validation and authorisation are obtained declaratively. The trade-off is conformity to Filament's conventions.
Spatie permissions	Hard-coded role checks	Roles and permissions become runtime data, editable without deployment. Enables the catalogue-driven design described in Section 8.
Spatie translatable	Per-locale tables	JSON per-locale attributes allow new languages without schema migration, and let editors maintain all locales from one form.
Tailwind CSS	Bootstrap or bespoke CSS	Utility classes with logical properties make right-to-left mirroring largely automatic — decisive for Dari and Pashto support.
Delegated donations	Native gateway	No payment data enters the system, minimising compliance burden. Control over the donation experience is ceded to the provider.
No public API	REST or GraphQL	Removes authentication, versioning and rate-limiting surface area. Laravel Sanctum can be introduced later without restructuring the domain model.
Docker Compose	Native installation	Guarantees environment parity across contributor workstations and mirrors the production runtime.
SQLite in development	MySQL everywhere	Zero-configuration onboarding and fast test execution. Requires discipline to avoid engine-specific constructs.

COHERENCE ASSESSMENT

The selections form a mutually reinforcing set. Laravel, Livewire, Filament, Blade, Alpine and Tailwind constitute a single well-integrated ecosystem with shared conventions and a common release cadence, minimising integration friction and easing the onboarding of engineers already familiar with the Laravel ecosystem.

SECTION 16

Technical Debt Register & Roadmap

The following register records known items requiring attention, ordered by priority. Recording them explicitly is a deliberate engineering practice: each item is understood, bounded and scheduled rather than latent.

TABLE 16.1 — REMEDIATION REGISTER

Ref	Item	Priority	Recommended action
TD-01	Duplicate Tailwind delivery	High	Retire the Play CDN reference and serve compiled Tailwind through the Vite manifest, migrating brand tokens into the CSS-first configuration.
TD-02	Environment file in version control	High	Remove the committed environment file from tracking, purge it from history and rotate any values it exposed.
TD-03	Inline Compose credentials	Medium	Relocate database passwords into an untracked environment file referenced by the Compose manifest.
TD-04	No continuous integration	High	Introduce a pipeline executing the test suite, Pint verification and a container build on every push and pull request.
TD-05	Narrow test coverage	Medium	Extend feature coverage to controllers, submission workflows and the expense import service; add regression tests for survival-rate computation.
TD-06	No error monitoring	Medium	Integrate an exception-tracking and performance-monitoring service before production launch.
TD-07	Placeholder project README	Medium	Replace the framework template with project-specific setup, architecture and contribution guidance.
TD-08	Absent JavaScript lock file	Medium	Commit the package lock file to guarantee reproducible front-end builds.
TD-09	Container port declaration	Low	Reconcile the image's exposed port with the Apache listener and the Compose port mapping.
TD-10	Unreferenced legacy code	Low	Remove superseded administrator middleware and the unrouted media controller.
TD-11	Static analysis absent	Low	Adopt a static analyser to detect type and nullability defects ahead of runtime.
TD-12	Uncommunicative commit history	Low	Adopt a conventional commit standard to render history auditable.

Suggested sequencing

Phase 1 — Pre-launch

TD-01, TD-02, TD-04. Resolve production styling delivery, secret exposure and automated verification.

Phase 2 — Stabilisation

TD-03, TD-05, TD-06, TD-08. Harden configuration, broaden coverage and gain production visibility.

Phase 3 — Maintainability

TD-07, TD-09 through TD-12. Improve documentation, remove dead code and raise static guarantees.

OVERALL ASSESSMENT

No item in this register represents an architectural defect. Every entry is a configuration, tooling or coverage matter addressable without structural change. The foundation — the domain model, the authorisation architecture and the localisation strategy — is sound.

APPENDIX A

Complete Dependency Manifest

A.1 PHP production dependencies (declared)

Package	Constraint	Resolved	Purpose
`php`	`^8.2`	–	Language runtime
`laravel/framework`	`^12.0`	`12.35.1`	Application framework
`filament/filament`	`^5.2`	`5.2.0`	Administration panel
`spatie/laravel-permission`	`^8.3`	`8.3.0`	Role-based access control
`spatie/laravel-translatable`	`^6.11`	`6.11.4`	Translatable model attributes
`phpoffice/phpspreadsheet`	`^5.5`	`5.5.0`	Spreadsheet import
`laravel/tinker`	`^2.10.1`	`2.10.1`	Interactive console

A.2 Significant transitive dependencies

Package	Resolved	Purpose
`livewire/livewire`	`4.1.4`	Reactive server-driven components
`filament/forms, tables, actions`	`5.2.0`	Form, table and action builders
`filament/infolists, schemas`	`5.2.0`	Read views and schema definitions
`filament/notifications, widgets`	`5.2.0`	Notifications and dashboard widgets
`filament/support`	`5.2.0`	Shared panel infrastructure
`blade-ui-kit/blade-heroicons`	`2.6.0`	SVG icon components
`guzzlehttp/guzzle`	`7.10.0`	HTTP client
`nesbot/carbon`	`3.10.3`	Date and time handling
`symfony/console`	`7.3.4`	Console command layer
`monolog/monolog`	`3.9.0`	Logging
`league/commonmark`	`2.7.1`	Markdown rendering
`ueberdosis/tiptap-php`	`2.1.0`	Rich-text document handling
`openspout/openspout`	`4.28.5`	Streaming spreadsheet reader
`pragmarx/google2fa`	`9.0.0`	Available for two-factor authentication; not currently enabled

A.3 PHP development dependencies

Package	Constraint	Resolved	Purpose
`phpunit/phpunit`	`^11.5.3`	`11.5.42`	Test framework
`laravel/pint`	`^1.24`	`1.25.1`	Code formatter
`laravel/sail`	`^1.41`	`1.46.0`	Docker harness
`laravel/pail`	`^1.2.2`	`1.2.3`	Log viewer
`mockery/mockery`	`^1.6`	`1.6.12`	Mocking library
`nunomaduro/collision`	`^8.6`	`8.8.2`	Console error reporting
`fakerphp/faker`	`^1.23`	`1.24.1`	Fixture generation

A.4 JavaScript dependencies

Package	Constraint	Purpose
`vite`	`^7.0.7`	Asset bundler and development server
`laravel-vite-plugin`	`^2.0.0`	Laravel manifest integration
`tailwindcss`	`^4.0.0`	Utility-first CSS framework
`@tailwindcss/vite`	`^4.0.0`	Tailwind CSS-first build plugin
`axios`	`^1.11.0`	Promise-based HTTP client
`concurrently`	`^9.0.1`	Parallel process orchestration

Resolved versions are not recorded, as no package lock file is committed. See remediation item TD-08.

APPENDIX B

Glossary of Terms

Term	Definition
Blade	Laravel's server-side templating engine, compiled once to plain PHP and cached.
Eloquent	Laravel's active-record object-relational mapper.
Filament	A component-driven administration panel framework for Laravel.
Livewire	A library providing reactive interfaces with state held on the server.
Alpine.js	A minimal JavaScript library for declarative behaviour expressed in markup.
Migration	A version-controlled, executable description of a database schema change.
Seeder	A class that populates the database with reference or demonstration data.
Policy	A class encapsulating authorisation rules for a specific model.
Gate	Laravel's authorisation primitive; a callback determining whether an action is permitted.

Glossary of Terms (continued)

Term	Definition
RBAC	Role-Based Access Control — permissions granted to roles, roles assigned to users.
Guard	A named authentication configuration defining how users are identified and stored.
Middleware	A layer intercepting HTTP requests before or after they reach the application.
Monolith	An architecture in which all functionality is delivered as a single deployable unit.
SPA	Single-Page Application — a client-rendered application; deliberately not used here.
SSR	Server-Side Rendering — HTML produced on the server, as employed throughout.
RTL / LTR	Right-to-left and left-to-right writing directions.
CSRF	Cross-Site Request Forgery; mitigated by per-session request tokens.
HMR	Hot Module Replacement — live asset updates during development.
Composer	The dependency manager for PHP projects.
Vite	A modern front-end build tool and development server.
Artisan	Laravel's command-line interface.

End of document

Every Tree for Hope — Technology Stack & System Architecture Specification
Document reference ETFH-TS-001 · Version 1.0 · 20 July 2026